

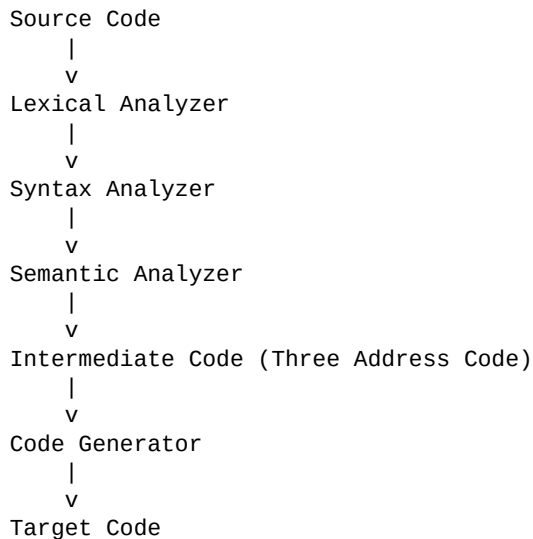
Compiler Design – Unit 1

Notes by Debkanta Maity – Debkanta Study Zone

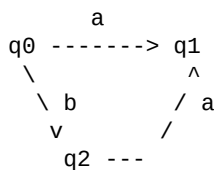
Topics covered (cover page):

- Lex, Parse, Analyze, Generate, Execute

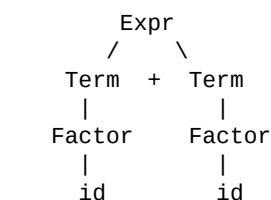
Compiler phases (cover diagram):



Sample finite automaton (cover diagram):



Sample expression grammar and parse tree (cover diagram):



$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

Introduction to Compiler

Compiler:

A compiler is a special program that translates code written in a high-level programming language (like C, C++, Java) into machine code (binary code) that a computer's processor can execute.

- It takes the entire program as input.
- Performs lexical, syntactic, and semantic analysis.
- Generates intermediate code and optimizes it.
- Finally produces executable machine code.

Advantages of a Compiler:

1) Fast Program Execution:

Once compiled, the program runs very fast because it is already translated into machine code.

2) Efficient Code Optimization:

Compilers optimize code during the compilation process for better performance and lower memory usage.

3) Early Error Detection:

Syntax and semantic errors are detected during compilation, before execution begins.

4) No Need for Source Code at Runtime:

The executable file can run independently without requiring the original source code or the compiler.

5) Security of Source Code:

The source code is not shared with users; only the executable file is distributed.

Disadvantages of a Compiler:

1) Slower Initial Compilation:

Compiling the entire program can take time, especially for large codebases.

2) Harder Debugging:

Debugging is difficult since the compiler displays all errors at once, and only after the full compilation.

3) No Partial Execution:

Errors must be corrected and the whole program recompiled before any part can be run.

4) Platform Dependency:

The machine code generated is platform-specific (e.g., Windows executable won't run on Linux).

5) Larger Executable Files:

Optimized machine code can sometimes lead to large executable files depending on the program.

Functions of a Compiler:

1. **Lexical Analysis:** Breaks code into tokens.
 2. **Syntax Analysis:** Analyzes grammar/syntax.
 3. **Semantic Analysis:** Checks meaning/logic.
 4. **Code Optimization:** Improves performance.
 5. **Code Generation:** Produces machine code.
 6. **Error Handling:** Detects and reports errors.
-

Types of a Compiler

1. Traditional Compiler
2. Cross Compiler
3. Just-In-Time (JIT) Compiler
4. Incremental Compiler
5. Source-to-Source Compiler (Transpiler)
6. Interpreter
7. Bootstrapping Compiler
8. Ahead-of-Time (AOT) Compiler

1. Traditional Compiler

Definition: A traditional compiler is a program that translates the entire high-level source code (like C or C++) into machine code before the program is executed.

Working: It performs all the phases of compilation in order:

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Intermediate Code Generation
5. Code Optimization
6. Machine Code Generation

Execution: Once compiled, the generated machine code (executable file) can be run independently without needing the compiler again.

Examples:

- GCC (GNU Compiler Collection) for C/C++
- Turbo C, Clang

Use Cases: System software, embedded systems, large-scale application.

2. Cross Compiler

Definition: A cross compiler is a type of compiler that generates executable code for a platform other than the one on which the compiler is running.

Working:

1. The cross compiler runs on host system (development system).
2. It produces code for a target system (execution system) with a different architecture or OS.
3. It translates source code written in high-level language into machine code compatible with the target device.

Execution:

1. Write source code on the host system.
2. Use the cross compiler to compile it.
3. The output is an executable binary for the target system.
4. The binary is then transferred and run on the target device (e.g., embedded systems, mobile devices).

Examples:

- A cross compiler like arm-none-eabi-gcc (for ARM architecture) runs on a Windows/Linux PC but generates code for ARM-based microcontrollers.
- Compiling Android apps on a PC to run on Android phones (ARM) is another real-world example.

Use Cases:

1. Embedded systems development (e.g., Arduino, Raspberry Pi).
2. Mobile app development (Android/iOS).
3. Game development (cross-platform games).

3. Just-In-Time (JIT) Compiler

Definition: A Just-In-Time (JIT) compiler is a part of the runtime environment that compiles code at execution time, converting intermediate code (like bytecode) into native machine code just before it runs.

Working:

1. Source code is first compiled into intermediate code (e.g., bytecode in Java or C#)
2. When the program runs, the JIT compiler translates the bytecode to native machine code on the fly.
3. This code is cached, so it runs faster if executed again.

Execution:

1. Write source code -> compiled into bytecode (intermediate form)
2. JVM or CLR (runtime environment) loads the bytecode.
3. JIT compiler translates frequently used bytecode into machine code.

4. Machine code runs directly on the CPU for faster performance.

Example:

1. Java JIT Compiler in JVM (Java Virtual Machine)
2. .NET JIT Compiler in CLR (Common Language Runtime).
3. V8 Engine in Google Chrome for JavaScript JIT compilation.

Use Cases:

1. Java applications (desktop, web, Android).
2. .NET applications (C#, ASP.NET).
3. Web browsers (JavaScript execution).

4. Incremental Compiler

Definition: An incremental compiler is a type of compiler that recompiles only the modified parts of a program instead of the whole source code, speeding up the compilation process.

Working:

1. The compiler tracks changes in the source code.
2. When a change is made, it only recompiles the affected modules or files.
3. It reuses the previously compiled code that hasn't changed.

Execution:

1. Initial compilation of the full source code.
2. Developer makes small changes to the code.
3. Incremental compiler detects and recompiles only changed parts.
4. Final executable is updated quickly using both new and reused compiled parts.

Example:

1. Eclipse IDE for Java uses incremental compilation.
2. IntelliJ IDEA and Android Studio also apply incremental compilation.

Use Cases:

1. Large software projects where full recompilation takes a long time.
2. IDE development tools for live code updates.

5. Source-to-Source Compiler (Transpiler)

Definition: A source-to-source compiler (also called a transcompiler or transpiler) is a type of compiler that translates source code from one high-level programming language to another high-level language.

Working:

1. Takes source code written in one programming language.
2. Converts it into equivalent code in another high-level language.
3. Maintains the logic and structure of the original code while adjusting for syntax and language features.

Execution:

1. Developer writes source code in Language A.
2. The source-to-source compiler translates it to Language B.
3. The translated code can then be compiled or interpreted normally by Language B's tools.
4. Final executable or output is produced using Language B's compiler.

Example:

```
TypeScript -> JavaScript
C++ -> C
CoffeeScript -> JavaScript
Babel (Used to convert modern JavaScript ES6+ ->
      older JavaScript for browser compatibility).
```

Use Cases:

1. Cross-platform development.
2. Optimization by converting high-level code to a more efficient high-level language.
3. Educational tools for teaching language translation.

6. Interpreter

Definition: An interpreter is a program that executes source code line-by-line or statement-by-statement without converting it into machine code beforehand.

Working:

1. Reads the source code directly.
2. Parses and analyzes each line at runtime.
3. Executes each instruction immediately after interpretation.
4. No intermediate executable file is created.

Execution:

1. Source code is written in a high-level language.
2. Each instruction is executed immediately.
3. The interpreter reads and interprets one line or block at a time.
4. No precompiled output is produced.

Example:

1. Python Interpreter
2. JavaScript Engines
3. Ruby, PHP

Use Cases:

1. Scripting and automation.
2. Rapid prototyping and debugging, since code can run without compilation.
3. Web development, especially JavaScript in browsers.
4. Educational tools, where immediate feedback is helpful.

7. Bootstrapping Compiler

Definition: A bootstrapping compiler is a compiler that is written in the same programming language that it is intended to compile. This process is called bootstrapping.

Working:

1. Initially, a simple version of the compiler is written in another language (like assembly or C).
2. This version is used to compile the more advanced version of the compiler written in its own language.
3. Eventually, the compiler becomes self-hosting (able to compile its own source code).

Execution:

1. Write a basic version of the compiler in a different language.
2. Use it to compile a more complex version written in the target language.
3. Gradually replace old versions until the compiler can compile itself.

Example:

1. The C compiler (gcc) was bootstrapped using an earlier C version.
2. The Go language compiler was initially written in C, then rewritten in Go itself.

Use Cases:

1. Developing new programming languages.
2. Creating self-hosting compilers.

3. Improving and evolving compilers incrementally.

8. Ahead-of-Time (AOT) Compiler

Definition: An Ahead-of-Time (AOT) compiler is a compiler that translates high-level source code or intermediate code into native machine code before program execution, i.e., at compile time.

Working:

1. Takes source code or intermediate code (like bytecode).
2. Compiles it into machine code for the target platform before the application runs.
3. Eliminates the need for runtime compilation (unlike JIT).

Execution:

1. Write source code -> compile to intermediate representation (e.g., bytecode).
2. AOT compiler converts bytecode to native machine code ahead of runtime.
3. The compiled binary is executed directly by the CPU, leading to faster startup and lower runtime overhead.

Example:

1. C and C++ compilers like Gcc and Clang.
2. Java AOT Compiler (JAOTC) introduced in JDK9.
3. .NET Native

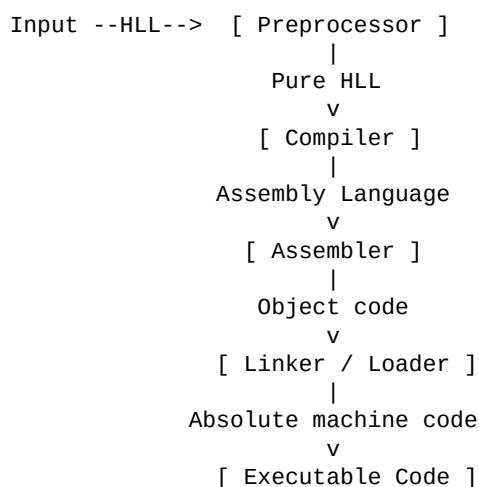
Use Cases:

1. Web frameworks.
 2. Low-memory or slow-CPU environments, where avoiding runtime compilation is beneficial.
 3. Real-time systems where predictable performance is required.
-

Language Processing System

A language processing system in compiler design is a comprehensive set of software tools that work in concert to transform a program written in a human-readable, high-level programming language (the source code) into a low-level form (typically machine code or assembly code) that a computer's processor can directly understand and execute. This intricate process involves multiple, sequential phases, each with a specialized role.

Components & Their Roles:



1. Preprocessor:

Input: Source code with directives (e.g., #include, #define).

Function: Handles macro expansion, file inclusion, conditional compilation, and comment removal.

Output: Expanded source code.

2. Compiler: (The core translator, typically in multiple phases)

A. Front-End (Analysis):

Lexical Analysis (Scanner):

- Reads characters and groups them into tokens (keywords, identifiers, operators, etc).
- Removes whitespace and comments.
- Reports lexical errors.

Output: Stream of tokens.

Syntax Analysis (Parser):

- Takes token stream and checks if it forms a grammatically correct program (per language's CFG)
- Builds a Parse Tree or Abstract Syntax Tree (AST) representing code structure.
- Reports syntax errors.

Output: Parse Tree / AST.

Semantic Analysis:

- Checks the "meaning" and consistency of program.
- Performs type checking (e.g., compatible types in operations).
- Checks variable declarations and scope.
- Populates/uses the Symbol Table (stores info about identifiers)
- Reports semantic errors.

Output: Annotated AST or validated intermediate representation.

B. Back-End (Synthesis):

Intermediate Code Generation:

- Converts the validated code into a machine-independent Intermediate Representation (IR).
- Simplifies subsequent optimization and portability.
- Common IR: Three-Address Code

Output: Intermediate Code.

Code Optimization:

- Analyzes and transforms IR to improve performance (speed, memory).
- Includes machine-independent (e.g., common subexpression elimination, dead code removal) and machine-dependent optimizations (e.g., register allocation, instruction scheduling).

Output: Optimized Intermediate Code.

Code Generation:

- Translates optimized IR into target machine's assembly language or machine code.
- Handles instruction selection, register allocation, and final instruction scheduling.

Output: Target Assembly Code or Machine Code.

3. Assembler:

Input: Assembly language code (from compiler).

Function: Translates assembly instructions into relocatable machine code.

Output: Object file (.o/.obj).

4. Linker:

Input: Multiple object files and library files.

Function: Combines object files and resolves external references (e.g., function calls between files, library functions).

Output: Executable File (.exe/no extension).

5. Loader:

Input: Executable files.

Function: Part of the OS; loads the executable into memory, allocates resources, and prepares it for execution.

Output: Program loaded in memory, ready to run.

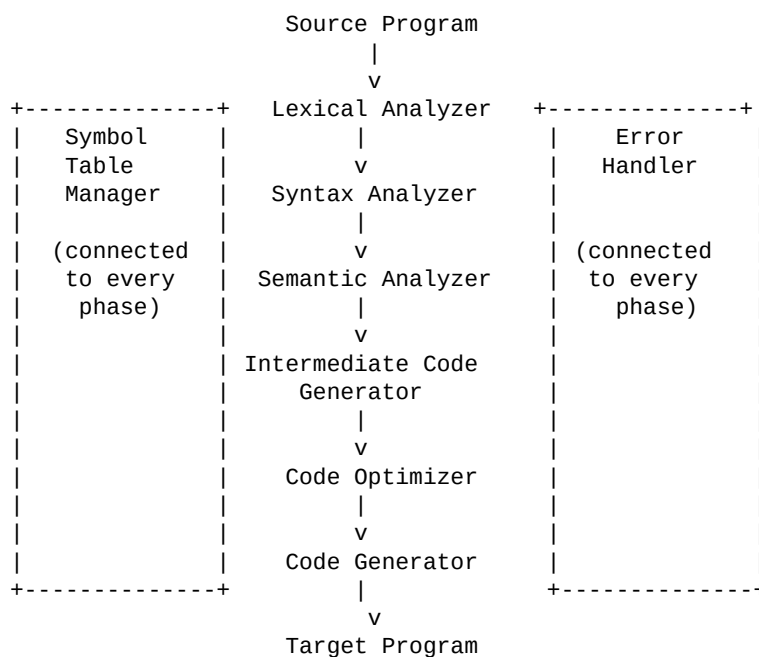
6. Executable Code:

It is low-level and machine-specific code that the machine can easily understand. Once the job of the linker and loader is done the object code is finally converted into executable code.

Key Supporting Components:

- **Symbol Table:** Central data structure storing information about identifiers.
 - **Error Handler:** Detects, reports, and sometimes recovers from errors during compilation.
-

The Phases of a Compiler



Worked Example 1: Price := amount + rate * 50

Lexical Analyzer output:

```
id1 := id2 + id3 * 50
```

Syntax Analyzer (parse tree):

```
:=
  id1
  +
  id2
  *
  id3
  50
```

Semantic Analyzer (annotated tree – 50 converted to real):

```
:=
  id1
```

```

+
  id2
  *
    id3
    inttoreal(50)

```

Intermediate Code Generator:

```

temp1 := inttoreal(50)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1    := temp3

```

Code Optimizer:

```

temp1 := id3 * 50.0
id1    := id2 + temp1

```

Code Generator (target assembly):

```

MOVF R2, id3
MULF R2, #50.0
MOVF R1, id2
ADDF R1, R2
MOVF id1, R1

```

Symbol Table:

#	Name
1	Price
2	amount
3	rate
4	

Worked Example 2: $a = x + x * y * z / 100$

Symbol Table:

Variable	Identifier / Attribute
a	Id1 * Att
x	Id2 * Att
y	Id3 * Att
z	Id4 * Att

Lexical Analyzer – token classification:

```

a: Identifier
=: Assignment OPR
x: Identifier
y: Identifier
z: Identifier
*: Arithmetic OPR
+: Arithmetic OPR
/: Arithmetic OPR
100: number/constant

```

Lexical Analyzer output:

```

Id1 = Id2 + Id2 * Id3 * Id4 / 100

```

Syntax Analyzer (parse tree):

```

=
  id1

```

```

+
  id2
  /
    *
      *
        id2
        id3
        id4
        100

```

Semantic Analyzer (annotated tree – 100 converted to float):

```

=
  id1
  +
    id2
    /
      *
        *
          id2
          id3
          id4
          inttfloat(100)

```

Intermediate Code Generator:

```

t1 = inttfloat(100)
t2 = id2 * id3
t3 = t2 * id4
t4 = t3 / t1
t5 = t4 + id2
id1 = t5

```

Code Optimizer:

```

t1 = id2 * id3
t2 = t1 * id4
t3 = t2 / 100.0
id1 = id2 + t3

```

Code Generator (target assembly):

```

MOVF id2, R1
MOVF id3, R2
MULF R2, R1
MOVF id4, R3
MULF R2, R3
DIVF R2, #100.0
ADDF R1, R2
MOVF R1, id1

```

Worked Example 3: $i = i * 70 + j + 2$

Symbol Table:

#	Name
1	id (i)
2	id (j)

Lexical Analyzer output:

```
id1 = id1 * 70 + id2 + 2
```

Syntax Analyzer (parse tree):

```

=
  id1
  +
    +
      *
        id1

```

```
      70
    id2
  2
```

Semantic Analyzer (annotated tree – 70 and 2 converted to float):

```
=
  id1
  +
  +
  *
    id1
    inttofloat(70)
  id2
  inttofloat(2)
```

Intermediate Code Generator:

```
t1 = inttofloat(70)
t2 = id1 * t1
t3 = t2 + id2
t4 = inttofloat(2)
t5 = t3 + t4
id1 = t5
```

Code Optimizer:

```
t1 = id1 * 70.0
t2 = t1 + id2
t3 = t2 + 2.0
id1 = t3
```

Code Generator (target assembly):

```
LDF  R1, id1
MULF R1, R1, #70.0
LDF  R2, id2
ADDF R1, R1, R2
ADDF R1, R1, #2.0
STF  id1, R1
```